Rotaciones y Formalismo Denavit-Hartenberg

Cinemática Directa e Inversa de Robots Seriales

Prof. Alberto Muñoz
amunoz@tec.mx

TE3001B · Tecnológico de Monterrey

Índice

1. Rotaciones en el Espacio 3D
2. Motivación y Convención DH
3. Los 4 Parámetros DH
4. Matriz de Transformación Homogénea DH
5. Cinemática Directa: Pseudocódigo y Python
6. Cinemática Inversa Analítica
7. Cinemática Inversa Numérica
8. Singularidades y el Jacobiano
9. Resumen y Conexiones

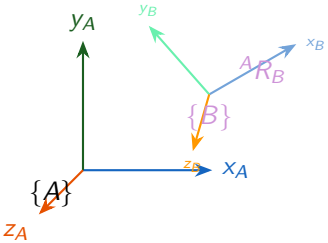
¿Por qué necesitamos describir rotaciones?

Problema central en robótica

- La posición de un cuerpo rígido en $\mathbb{R}^3$ requiere **6 grados de libertad**
- 3 para la **traslación**: (x, y, z)
- 3 para la **orientación**: ¿cómo representarla?
- Cada efector final tiene una orientación que importa (pinza, soldadura, cirugía)

Marco de referencia

Un **marco** $\{A\}$ es un sistema de ejes ortonormales dextrógiros $(\hat{x}, \hat{y}, \hat{z})$ con origen fijo. La orientación de $\{B\}$ respecto a $\{A\}$ queda completamente descrita por una **matriz de rotación** $R \in SO(3)$.



$${}^AR_B \in \mathbb{R}^{3\times 3} \text{ convierte vectores de } \{B\} \text{ a } \{A\}\text{: } {}^A\mathrm{v} = {}^AR_B\ {}^B\mathrm{v}$$

El grupo $SO(3)$: propiedades fundamentales

Definición formal:

$$SO(3) = \{ R \in \mathbb{R}^{3\times 3} \mid R^\top R = I,\ \det(R) = +1 \}$$

Propiedades esenciales

- $\det(R) = +1$ (orientación preservada)
- $R^{-1} = R^\top$ (inversa = transpuesta)
- $R^\top R = RR^\top = I_3$
- Las **columnas** son vectores unitarios ortonormales: $\hat{x}_B, \hat{y}_B, \hat{z}_B$ expresados en $\{A\}$
- Las **filas** son $\hat{x}_A, \hat{y}_A, \hat{z}_A$ expresados en $\{B\}$
- $SO(3)$ es un grupo: componer rotaciones da otra rotación

Columnas = ejes del marco rotado

$$^{A}R_B = \begin{bmatrix} \hat{x}_B & \hat{y}_B & \hat{z}_B \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

Dimensión del espacio

$SO(3)$ tiene **3 grados de libertad** (no 9), ya que $R^\top R = I$ impone 6 restricciones a los 9 elementos de la matriz.

Composición de rotaciones

$^{A}R_C = {}^{A}R_B \cdot {}^{B}R_C$

La multiplicación **no** es conmutativa en general.

Rotaciones elementales alrededor de los ejes principales

Las tres rotaciones básicas de ángulo θ alrededor de x, y y z:

$R_x(\theta)$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & c\theta & -s\theta \\ 0 & s\theta & c\theta \end{bmatrix}$$

Rota el plano YZ

$R_y(\theta)$

$$\begin{bmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{bmatrix}$$

Rota el plano XZ

$R_z(\theta)$

$$\begin{bmatrix} c\theta & -s\theta & 0 \\ s\theta & c\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Rota el plano XY

Verificación inmediata

- $\det(R_i(\theta)) = \cos^2\theta + \sin^2\theta = 1$ ✓
- $R_i(\theta)^\top = R_i(-\theta)$: invertir = cambiar signo
- $R_i(0) = I_3$: ángulo cero = identidad ✓
- $R_i(\alpha)R_i(\beta) = R_i(\alpha+\beta)$: ángulos del mismo eje se

Notación abreviada

$c\theta = \cos\theta$, $s\theta = \sin\theta$

Se usa en toda la literatura de robótica y en la matriz DH que veremos después.

Composición de rotaciones: el orden importa

No conmutatividad — ejemplo numérico:
Sea $R_z(90°)$ y $R_x(90°)$:

$$R_z R_x = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

$$R_x R_z = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix}$$

$\Rightarrow R_z R_x \neq R_x R_z$ **El orden es crítico.**

Convención de multiplicación

- **Rotaciones intrínsecas** (marco móvil):
  se multiplica a la *derecha*
- **Rotaciones extrínsecas** (marco fijo):
  se multiplica a la *izquierda*
- En DH: todas las transformaciones son **locales al marco actual** $\Rightarrow$ multiplicación a la derecha

$$^{A}R_C = {}^{A}R_B \cdot {}^{B}R_C$$

Leer de **derecha a izquierda**:
primero $B \to C$, luego $A \to B$.

Ángulos de Euler y convención RPY

Ángulos de Euler ZYX (convención aeroespacial):

$$R = R_z(\psi)\, R_y(\theta)\, R_x(\phi)$$

- ϕ (*roll*): giro alrededor de x
- θ (*pitch*): giro alrededor de y'
- ψ (*yaw*): giro alrededor de z''

Extracción de RPY desde R:

$$\phi = \operatorname{atan2}(R_{32},\, R_{33})$$

$$\theta = \operatorname{atan2}(-R_{31},\, \sqrt{R_{32}^2 + R_{33}^2})$$

$$\psi = \operatorname{atan2}(R_{21},\, R_{11})$$

Singularidad de Gimbal Lock

Cuando $\theta = \pm 90°$: $\cos\theta = 0$,
los ejes x y z se alinean y se pierde 1 DOF.

La solución es usar **cuaterniones unitarios** o el **eje-ángulo**.

Otras convenciones frecuentes

- **ZYZ** (Euler clásico): robótica industrial
- **XYZ**: ROS `tf` (rotaciones extrínsecas)
- **ZYX** (RPY): aviación, ROS URDF
- Las 12 convenciones son equivalentes; *hay que especificar siempre cuál se usa.*

Representación eje-ángulo

Teorema de Euler: cualquier rotación en $\mathbb{R}^3$ puede describirse como una **rotación de ángulo** θ **alrededor de un eje unitario** $\hat{k}$.
Fórmula de Rodrigues:

$$R = I + \sin\theta\,[\hat{k}]_\times + (1-\cos\theta)\,[\hat{k}]_\times^2$$

donde el operador *skew-symmetric*:

$$[\hat{k}]_\times = \begin{bmatrix} 0 & -k_z & k_y \\ k_z & 0 & -k_x \\ -k_y & k_x & 0 \end{bmatrix}$$

cumple $[\hat{k}]_\times \mathrm{v} = \hat{k} \times \mathrm{v}$.

Extracción de eje-ángulo desde R

$$\theta = \arccos\left(\frac{\mathrm{tr}(R)-1}{2}\right)$$

$$\hat{k} = \frac{1}{2\sin\theta}\begin{bmatrix} R_{32}-R_{23} \\ R_{13}-R_{31} \\ R_{21}-R_{12} \end{bmatrix}$$

Conexión con cuaterniones

El cuaternión unitario:

$$\mathrm{q} = \left(\cos\tfrac{\theta}{2},\ \hat{k}\sin\tfrac{\theta}{2}\right)$$

evita el gimbal lock y es compacto (4 escalares). Muy usado en ROS (`tf2`).

Python – Rotaciones elementales y composición

```
import numpy as np
def Rx(t): ct,st=np.cos(t),np.sin(t); return np.array([[1,0,0],[0,ct,-st],[
def Ry(t): ct,st=np.cos(t),np.sin(t); return np.array([[ct,0,st],[0,1,0],[-
def Rz(t): ct,st=np.cos(t),np.sin(t); return np.array([[ct,-st,0],[st,ct,0]
# Composicion ZYX (RPY): yaw=psi, pitch=theta, roll=phi
def rpy_to_R(roll, pitch, yaw):
    return Rz(yaw) @ Ry(pitch) @ Rx(roll)
R = rpy_to_R(np.radians(10), np.radians(20), np.radians(30))
print("det(R)  =", round(np.linalg.det(R), 10))    # debe ser 1
print("R^T R:\n", (R.T @ R).round(8))               # debe ser I
# Extraer RPY desde R
pitch = np.arctan2(-R[2,0], np.sqrt(R[2,1]**2 + R[2,2]**2))
roll  = np.arctan2(R[2,1], R[2,2])
yaw   = np.arctan2(R[1,0], R[0,0])
print(f"Roll={np.degrees(roll):.2f}  Pitch={np.degrees(pitch):.2f}
Yaw={np.degrees(yaw):.2f}")
# Verificar no-conmutatividad
ang = np.radians(90)
print("Rz*Ry == Ry*Rz?", np.allclose(Rz(ang)@Ry(ang), Ry(ang)@Rz(ang)))
```

Python – Fórmula de Rodrigues y eje-ángulo

```python
import numpy as np
def skew(k):
    """Operador skew-symmetric de un vector 3D."""
    return np.array([[0,-k[2],k[1]],[k[2],0,-k[0]],[-k[1],k[0],0]])
def rodrigues(k_hat, theta):
    """Rotacion de angulo theta alrededor del eje unitario k_hat (Rodrigues).
    K = skew(k_hat)
    return np.eye(3) + np.sin(theta)*K + (1-np.cos(theta))*(K @ K)
def R_to_axis_angle(R):
    """Extrae eje y angulo desde una matriz de rotacion."""
    theta = np.arccos(np.clip((np.trace(R)-1)/2, -1, 1))
    if abs(np.sin(theta)) < 1e-8:
        return np.array([0,0,1]), 0.0    # identidad
    k = (1/(2*np.sin(theta))) * np.array([R[2,1]-R[1,2],
                                          R[0,2]-R[2,0],
                                          R[1,0]-R[0,1]])
    return k, theta
# Demo: rotar 60 grados alrededor de (1,1,0)/sqrt(2)
k = np.array([1,1,0])/np.sqrt(2)
```

Transformaciones homogéneas: unir rotación y traslación

Un cuerpo rígido en 3D requiere describir *tanto* su orientación *como* su posición. La **matriz homogénea** $T \in SE(3)$ los unifica:

$$T = \begin{bmatrix} R_{3\times3} & \mathrm{p}_{3\times1} \\ 0_{1\times3} & 1 \end{bmatrix} \in \mathbb{R}^{4\times4}$$

Transforma un punto ${}^{B}\mathrm{p}$ del marco $\{B\}$ al marco $\{A\}$:

$$\begin{bmatrix} {}^{A}\mathrm{p} \\ 1 \end{bmatrix} = T \begin{bmatrix} {}^{B}\mathrm{p} \\ 1 \end{bmatrix} = \begin{bmatrix} R\,{}^{B}\mathrm{p} + \mathrm{p} \\ 1 \end{bmatrix}$$

Propiedades de $T \in SE(3)$

- **Inversa**:

$$T^{-1} = \begin{bmatrix} R^{\top} & -R^{\top}\mathrm{p} \\ 0 & 1 \end{bmatrix}$$

- **Composición**: ${}^{A}T_{C} = {}^{A}T_{B} \cdot {}^{B}T_{C}$
- **NO conmuta**: $T_1 T_2 \neq T_2 T_1$ en general
- $\det(T) = \det(R) = +1$

$SE(3) = SO(3) \ltimes \mathbb{R}^3$:
el grupo especial euclidiano es el producto semidirecto de rotaciones y traslaciones.

Python – Transformaciones homogéneas

```python
import numpy as np

def T_from_Rp(R, p):
    """Construye T in SE(3) desde R in SO(3) y traslacion p."""
    T = np.eye(4)
    T[:3,:3] = R
    T[:3, 3] = p
    return T
def T_inv(T):
    """Inversa de T in SE(3): aprovecha la estructura de bloque."""
    R, p = T[:3,:3], T[:3,3]
    T_i = np.eye(4)
    T_i[:3,:3] =  R.T
    T_i[:3, 3] = -R.T @ p
    return T_i
# Ejemplo: EF a 0.5 m adelante del origen, rotado 45 grados en z
Rz45 = np.array([[0.707,-0.707,0],[0.707,0.707,0],[0,0,1]])
p    = np.array([0.5, 0.2, 0.0])
T_AB = T_from_Rp(Rz45, p)
```

Resumen: representaciones de orientación

Representación	Parámetros	Singularidades	Compacta	Uso típico
Matriz $R \in SO(3)$	9 (6 restricciones)	No	No	Cálculo, DH
Ángulos RPY / Euler	3	Gimbal lock	Sí	URDF, visualización
Eje-ángulo $(\hat{k}, \theta)$	4 (1 restricción)	$\theta = 0, \pi$	Sí	Interpolación
Cuaternión unitario q	4 (1 restricción)	No	Sí	ROS `tf2`, IMU

Regla práctica

- **Cálculo analítico** (DH, FK, IK): usar R
- **Interfaz con el usuario**: RPY
- **Control y estimación**: cuaterniones
- **Interpolación suave (SLERP)**: cuaterniones

Conversión cuaternión $\leftrightarrow$ R

$$q = (q_0, q_1, q_2, q_3), \quad \|q\| = 1$$

La matriz correspondiente tiene entradas cuadráticas en q_0, q_1, q_2, q_3. Libreria: `pytransform3d`, `scipy.spatial.transform`.

¿Por qué necesitamos un formalismo sistemático?

Sin convención

- Cada diseño define marcos propios
- Imposible automatizar el cálculo
- Difícil reutilizar código
- Inconsistencias al integrar subsistemas

Historia

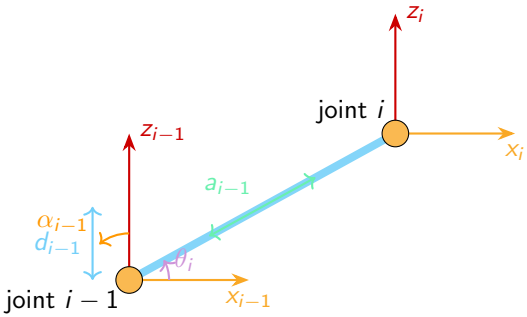
Jacques Denavit y Richard Hartenberg propusieron en 1955 (*Trans. ASME*) una representación con **solo 4 parámetros** por eslabón, válida para cualquier cadena cinemática serial.

Con DH (1955)

- ✓ 4 números describen *cualquier* eslabón
- ✓ Automatizable: tabla $\rightarrow$ matrices $\rightarrow$ código
- ✓ Estándar internacional (ISO, ROS, MATLAB)
- ✓ Válido para joints **R** (revoluta) y **P** (prismática)
- ✓ Base para FK, IK, Jacobiano y control

Idea central: reducir cualquier robot serial a una *tabla* de 4 columnas

Asignación de marcos de referencia



Reglas de asignación:

1. z_i a lo largo del eje de la joint $i+1$
2. x_i a lo largo de la **normal común** entre z_{i-1} y z_i
3. y_i completa el sistema dextrógiro
4. $\{0\}$: libre (generalmente z_0 hacia arriba)

Importante

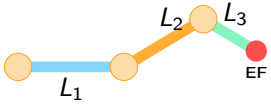
Los ejes z determinan los **ejes de joint**.
Los ejes x conectan normales comunes.

Los 4 parámetros de Denavit-Hartenberg

Símbolo	Nombre	Definición geométrica
θ_i	Ángulo de articulación	Rotación en torno a z_{i-1} para alinear x_{i-1} con x_i *Variable si joint R, constante si P*
d_i	Desplazamiento de articulación	Traslación a lo largo de z_{i-1} desde x_{i-1} hasta x_i *Variable si joint P, constante si R*
a_i	Longitud del eslabón	Distancia mínima entre z_{i-1} y z_i (normal común) *Siempre constante geométrica*
α_i	Torsión del eslabón	Rotación en torno a x_i para alinear

Tabla DH – Ejemplo: Robot 3R Planar

i	θ_i	d_i	a_i	α_i	Tipo
1	q_1	0	L_1	0	R
2	q_2	0	L_2	0	R
3	q_3	0	L_3	0	R



Procedimiento general

1. Identificar joints y eslabones
2. Colocar marcos $\{0\}$ a $\{n\}$
3. Medir $(\theta_i, d_i, a_i, \alpha_i)$
4. Completar la tabla
5. Calcular matrices ${}^{i-1}T_i$
6. Multiplicar: ${}^{0}T_n = \prod_{i=1}^{n} {}^{i-1}T_i$

Para robots planares

$\alpha_i = 0°$ para todas las joints
$\Rightarrow$ todos los ejes z son paralelos

La matriz homogénea DH: derivación

La transformación de eslabón a eslabón se descompone en **4 transformaciones elementales** aplicadas en orden fijo:

$$^{i-1}T_i = \underbrace{R_z(\theta_i)}_{\substack{\text{rotar en torno}\\ \text{a } z_{i-1}}} \cdot \underbrace{T_z(d_i)}_{\substack{\text{trasladar}\\ \text{en } z_{i-1}}} \cdot \underbrace{T_x(a_i)}_{\substack{\text{trasladar}\\ \text{en } x_i}} \cdot \underbrace{R_x(\alpha_i)}_{\substack{\text{rotar en torno}\\ \text{a } x_i}}$$

El orden es estricto e irreversible

Cada transformación se aplica respecto al *marco actual*, por lo que el orden $R_z \to T_z \to T_x \to R_x$ no puede alterarse.

Conexión con lo visto en rotaciones

$R_z(\theta_i)$ y $R_x(\alpha_i)$ son exactamente las **rotaciones elementales** estudiadas en la sección anterior. La novedad del formalismo DH es el *orden* prescrito en que se combinan

Multiplicando las cuatro matrices 4×4 se obtiene el resultado explícito que vemos en la siguiente diapositiva.

La matriz homogénea DH: resultado explícito

$$
{}^{i-1}T_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

Estructura de $T \in SE(3)$

$$
T = \begin{bmatrix} R_{3\times3} & \mathrm{p}_{3\times1} \\ 0_{1\times3} & 1 \end{bmatrix}
$$

- $R = R_z(\theta_i)\, R_x(\alpha_i)$: rotación
- $\mathrm{p} = [a_i c\theta_i,\ a_i s\theta_i,\ d_i]^\top$: traslación

Cinemática directa total

$$
{}^0T_n(q) = \prod_{i=1}^{n} {}^{i-1}T_i(q_i)
$$

La posición del efector final:

$$
\mathrm{p}_{EF} = {}^0T_n[0{:}3,\ 3]
$$

La orientación:

$$
R_{EF} = {}^0T_n[0{:}3,\ 0{:}3]
$$

Propiedades importantes de T

Propiedades de $R \in SO(3)$:

- $\det(R) = +1$
- $R^{-1} = R^\top$ (rotaciones son ortogonales)
- Columnas: vectores unitarios ortonormales

Propiedades de $T \in SE(3)$:

- $T^{-1} = \begin{bmatrix} R^\top & -R^\top \mathrm{p} \\ 0 & 1 \end{bmatrix}$
- Componer: ${}^0T_n = {}^0T_1 \cdots {}^{n-1}T_n$
- NO conmuta: ${}^0T_1 \cdot {}^1T_2 \neq {}^1T_2 \cdot {}^0T_1$

Caso planar ($\alpha_i = 0$)

La matriz se simplifica a:

$$ {}^{i-1}T_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i & 0 & a_i\sin\theta_i \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

Sólo 2 DOF efectivos en el plano XY.

Pseudocódigo – Cinemática Directa (FK)

Algoritmo FK con tabla DH

```
Entrada:  tabla DH de n eslabones,
q ∈ ℝ^n
Salida:   T_0^n ∈ SE(3)
1.  T ← I_4
2.  para i = 1 a n:
3.      θ ← θ_i^(DH) + q_i  (si joint R)
d ← d_i^(DH) + q_i  (si joint P)
4.      T_i ← DH(θ, d, a_i, α_i)
5.      T ← T · T_i
6.  fin para
7.  devolver T
```

Complejidad

- $O(n)$ multiplicaciones 4×4
- Siempre tiene solución única
- Sin singularidades propias
- Determinísta y eficiente

Extracción de resultados

$$\mathrm{p}_{EF} = T[0{:}3,\ 3]$$
$$R_{EF} = T[0{:}3,\ 0{:}3]$$
$$\phi_{EF} = \mathrm{atan2}(R_{21}, R_{11})$$

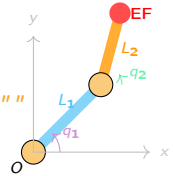
(para robots planares: $\phi = \sum q_i$)

Python – Función dh_matrix()

```python
import numpy as np
def dh_matrix(theta, d, a, alpha):
    """Matriz de transformacion homogenea DH 4x4.
    Args:
        theta : rotacion alrededor de z_{i-1}  [rad]
        d     : traslacion a lo largo de z_{i-1} [m]
        a     : longitud del eslabon             [m]
        alpha : torsion del eslabon              [rad]
    Returns:
        T : np.ndarray (4x4)
    """
    ct, st = np.cos(theta), np.sin(theta)
    ca, sa = np.cos(alpha), np.sin(alpha)
    T = np.array([
        [ct,  -st*ca,   st*sa,   a*ct],
        [st,   ct*ca,  -ct*sa,   a*st],
        [ 0,       sa,       ca,      d],
        [ 0,        0,        0,      1]
    ])
```

Python – FK Robot 2R Planar

```python
def fk_2r(q1, q2, L1=1.0, L2=1.0):
    """Cinematica directa robot 2R planar (DH)."""
    # Tabla DH: theta   d    a     alpha
    T01 = dh_matrix(q1, 0, L1,  0)
    T12 = dh_matrix(q2, 0, L2,  0)
    # Transformacion total base -> EF
    T02 = T01 @ T12
    px  = T02[0, 3]  # posicion x del EF
    py  = T02[1, 3]  # posicion y del EF
    phi = q1 + q2    # orientacion planar
    return px, py, phi, T02
# Ejemplo
q = np.radians([45, 30])
px, py, phi, T = fk_2r(q[0], q[1], L1=1.0, L2=
print(f"EF -> x={px:.4f}, y={py:.4f}
```



FK exacta 2R

$$p_x = L_1 c_1 + L_2 c_{12}$$
$$p_y = L_1 s_1 + L_2 s_{12}$$
$$\phi = q_1 + q_2$$

Python – FK Robot 3R Planar

```python
def fk_3r(q, L=[1.0, 0.8, 0.5]):
    """Cinematica directa robot 3R planar.
    q : array [q1, q2, q3] en radianes
    L : longitudes de eslabon [L1, L2, L3]
    """
    T = np.eye(4)
    params = [(L[0], 0), (L[1], 0), (L[2], 0)]  # (a_i, alpha_i)
    for i, (ai, alphai) in enumerate(params):
        Ti = dh_matrix(q[i], 0, ai, alphai)  # d=0, theta=q[i]
        T  = T @ Ti
    return T
# Test
q = np.radians([30, 45, -20])
T = fk_3r(q)
print("Posicion EF:", T[:3, 3].round(4))
print("Rotacion R:\n", T[:3, :3].round(4))
# Posicion alternativa (formula cerrada para 3R planar):
L = [1.0, 0.8, 0.5]
px = L[0]*np.cos(q[0]) + L[1]*np.cos(q[0]+q[1]) + L[2]*np.cos(q[0]+q[1]+q[2]
```

Planteamiento de la Cinemática Inversa

Problema

Dado $x_d \in \mathbb{R}^m$ (posición/orientación deseada), encontrar $q \in \mathbb{R}^n$ tal que:

$$f(q) = x_d$$

donde $f : \mathbb{R}^n \to \mathbb{R}^m$ es la FK.

Tres enfoques principales:

1. **Analítica / Geométrica**
   Solución en forma cerrada
2. **Numérica iterativa**
   Newton-Raphson, DLS
3. **Aprendizaje automático**
   Redes neuronales

	Analítica	**Numérica**
Velocidad	✓✓	✓
Generalidad	×	✓✓
Múltiples sol.	Explícitas	Según q_0
Singularidades	Detectables	Requiere DLS
Redundancia	No aplica	Pseudoinversa

Dificultad intrínseca

IK es generalmente *no lineal* y puede tener:

- Infinitas soluciones ($n > m$)
- Solución única
- Ninguna solución (q fuera del workspace)

IK Analítica: Robot 2R – Derivación completa

FK directa (robot 2R planar):

$$p_x = L_1 \cos q_1 + L_2 \cos(q_1 + q_2)$$
$$p_y = L_1 \sin q_1 + L_2 \sin(q_1 + q_2)$$

Paso 1 – Obtener q_2 por ley de cosenos:

$$d^2 = p_x^2 + p_y^2 = L_1^2 + L_2^2 + 2L_1L_2 \cos q_2$$
$$\Rightarrow \quad c_2 = \frac{p_x^2 + p_y^2 - L_1^2 - L_2^2}{2L_1L_2}$$
$$q_2 = \pm \arccos(c_2)$$

Paso 2 – Obtener q_1:

$$k_1 = L_1 + L_2 \cos q_2, \quad k_2 = L_2 \sin q_2$$
$$q_1 = \mathrm{atan2}(p_y,\ p_x) - \mathrm{atan2}(k_2,\ k_1)$$

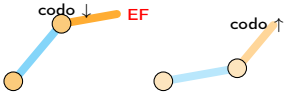
Dos soluciones

$+\arccos \Rightarrow$ **codo abajo**
$-\arccos \Rightarrow$ **codo arriba**

Condición de alcanzabilidad

$$|L_1 - L_2| \ \leq \ d \ \leq \ L_1 + L_2$$

Si no se cumple: punto inalcanzable.



Python – IK Analítica Robot 2R

```python
import numpy as np
def ik_2r_closed(px, py, L1=1.0, L2=1.0):
    """Cinematica inversa analitica robot 2R planar.
    Retorna dos soluciones: codo abajo y codo arriba.
    """
    d = np.sqrt(px**2 + py**2)
    # --- Verificar alcanzabilidad ---
    if d > (L1 + L2) or d < abs(L1 - L2):
        raise ValueError(f"Punto inalcanzable: d={d:.4f}")
    # --- Ley de cosenos: q2 ---
    cos_q2 = (d**2 - L1**2 - L2**2) / (2*L1*L2)
    cos_q2 = np.clip(cos_q2, -1, 1)    # robustez numerica
    q2_abajo  = +np.arccos(cos_q2)     # codo abajo
    q2_arriba = -np.arccos(cos_q2)     # codo arriba
    soluciones = []
    for q2 in [q2_abajo, q2_arriba]:
        k1 = L1 + L2*np.cos(q2)
        k2 = L2*np.sin(q2)
        q1 = np.arctan2(py, px) - np.arctan2(k2, k1)
```

IK Analítica: Robot SCARA (4-DOF)

Tabla DH – SCARA:

i	θ	d	a	α	Tipo
1	q_1	d_1	L_1	$0°$	R
2	q_2	0	L_2	$180°$	R
3	0	q_3	0	$0°$	P
4	q_4	d_4	0	$0°$	R

Solución analítica:

$$c_2 = \frac{p_x^2 + p_y^2 - L_1^2 - L_2^2}{2L_1L_2}$$

$$q_2 = \arccos(c_2)$$

$$q_1 = \mathrm{atan2}(p_y, p_x) - \mathrm{atan2}(k_2, k_1)$$

$$q_3 = d_1 - p_z - d_4$$

Características SCARA

- joints 1, 2, 4: **rotacionales** en eje Z (paralelas)
- joint 3: **prismática** (movimiento vertical)
- $\alpha_2 = 180° \Rightarrow$ brazo se dobla en plano horizontal
- Muy usado en ensamble y *pick-and-place*

Desacoplamiento

IK SCARA se desacopla naturalmente:

1. Plano horizontal: q_1, q_2 (igual que 2R)
2. Eje vertical: q_3
3. Orientación: q_4

Python – IK Analítica SCARA

```python
import numpy as np
def ik_scara(px, py, pz, phi, d1=0.3, L1=0.3, L2=0.25, d4=0.1):
    """Cinematica inversa analitica SCARA (4-DOF).
    Entradas: posicion (px,py,pz) y orientacion phi [rad]
    Salida:   q = [q1, q2, q4, d3]
    """
    # --- q2: ley de cosenos (plano horizontal) ---
    D = (px**2 + py**2 - L1**2 - L2**2) / (2*L1*L2)
    D = np.clip(D, -1, 1)           # robustez numerica
    q2 = np.arccos(D)               # codo abajo
    # --- q1: angulo base ---
    k1 = L1 + L2*np.cos(q2)
    k2 = L2*np.sin(q2)
    q1 = np.arctan2(py, px) - np.arctan2(k2, k1)
    # --- q4: orientacion del efector final ---
    q4 = phi - q1 - q2
    # --- d3: desplazamiento vertical (joint prismatica) ---
    d3 = d1 - pz - d4
    return np.array([q1, q2, q4, d3])
```

IK Numérica: Newton-Raphson y DLS

Motivación: la relación $\mathrm{x} = f(q)$ es *no lineal*. El Jacobiano la linealiza localmente:

$$f(q + \Delta q) \approx f(q) + J(q)\,\Delta q$$

Algoritmo iterativo:

1. Inicializar $q^{(0)}$
2. $\mathrm{e}^{(k)} = \mathrm{x}_d - f(q^{(k)})$
3. Si $\|\mathrm{e}^{(k)}\| < \varepsilon \Rightarrow$ **converge**
4. $\Delta q^{(k)} = J^{\dagger}\,\mathrm{e}^{(k)}$ (pseudoinversa)
5. $q^{(k+1)} = q^{(k)} + \alpha\,\Delta q^{(k)}$
6. Volver a 2

Pseudoinversa pura – problema

$$\Delta q = J^{+} e = J^{\top}(JJ^{\top})^{-1} e$$

En singularidades: $JJ^{\top} \to$ singular $\Rightarrow \|\Delta q\| \to \infty$

Damped Least Squares (DLS)

$$\boxed{\Delta q = J^{\top}(JJ^{\top} + \lambda^2 I)^{-1}\mathrm{e}}$$

- $\lambda > 0$: regularización
- $\lambda \to 0$: pseudoinversa pura
- Minimiza $\|\Delta q\|^2 + \lambda^2\|\mathrm{e}\|^2$
- Robusto cerca de singularidades

El Jacobiano analítico para el robot nR planar

Para el robot nR planar con eslabones $L_1, \ldots, L_n$:

$$p_x(q) = \sum_{i=1}^{n} L_i \cos\left(\sum_{j=1}^{i} q_j\right)$$

$$p_y(q) = \sum_{i=1}^{n} L_i \sin\left(\sum_{j=1}^{i} q_j\right)$$

El Jacobiano $J \in \mathbb{R}^{2\times n}$:

$$J_{1k} = \frac{\partial p_x}{\partial q_k} = -\sum_{i=k}^{n} L_i \sin\left(\sum_{j=1}^{i} q_j\right)$$

$$J_{2k} = \frac{\partial p_y}{\partial q_k} = \sum_{i=k}^{n} L_i \cos\left(\sum_{j=1}^{i} q_j\right)$$

Jacobiano 2R explícito

$$J = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix}$$

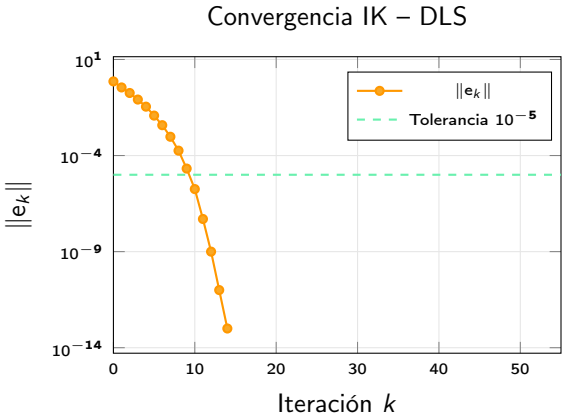
donde $c_{12} = \cos(q_1 + q_2)$, etc.

Singularidad del 2R

$$\det(J) = L_1 L_2 \sin q_2$$

$\Rightarrow$ singular cuando $q_2 = 0^\circ$ o 180°

Convergencia del método DLS



Parámetros del método

- ε: tolerancia (típico: 10^{-5} a 10^{-8} m)
- λ: damping factor (típico: 0.01 a 0.1)
- $q^{(0)}$: punto inicial (afecta a cuál solución converge)
- $k_{\max}$: iteraciones máximas (típico: 50–200)

Convergencia cuadrática

Cerca de la solución, el error se reduce *cuadráticamente* en cada iteración (igual que Newton-Raphson clásico).

Python – IK Numérica 3R con DLS

```python
import numpy as np

def jacobian_3r(q, L=[1.0, 0.8, 0.5]):
    """Jacobiano analitico 3R planar (posicion 2D)."""
    c1   = np.cos(q[0]);   c12  = np.cos(q[0]+q[1])
    c123 = np.cos(q[0]+q[1]+q[2])
    s1   = np.sin(q[0]);   s12  = np.sin(q[0]+q[1])
    s123 = np.sin(q[0]+q[1]+q[2])
    J = np.array([
        [-L[0]*s1 - L[1]*s12 - L[2]*s123,
         -L[1]*s12 - L[2]*s123, -L[2]*s123],
        [ L[0]*c1 + L[1]*c12 + L[2]*c123,
          L[1]*c12 + L[2]*c123,  L[2]*c123]
    ])
    return J

def ik_3r_dls(target, q0, L=[1.0,0.8,0.5],
              lam=0.05, max_iter=200, tol=1e-5):
    """IK numerica 3R con Damped Least Squares."""
```

Python – Uso y verificación de IK 3R

```python
# --- Demo IK 3R DLS ---
target = np.array([1.5, 0.8])
# posicion deseada [m]
q0     = np.radians([20, 30, -10])
# configuracion inicial

q_sol, iters, ok = ik_3r_dls(target, q0)

print(f"Convergencia: {ok}
({iters} iteraciones)")
print(f"q = {np.degrees(q_sol).round(3)} grados"

# Verificar con FK
T = fk_3r(q_sol)
print(f"EF = {T[:2,3].round(6)}")
print(f"Target = {target}")
print(f"Error  = {np.linalg.norm(T[:2,3]-target):.2e} m"
```

Salida esperada

```
Convergencia:  True (11 it.)
q = [28.4, 41.2, -12.1]
EF = [1.5, 0.8]
Error = 3.1e-07 m
```

Nota sobre q_0

Distintas condiciones iniciales pueden converger a **configuraciones diferentes** – todas válidas.

Singularidades: cuando la IK falla

Definición

Una **singularidad** ocurre cuando:

$$\text{rango}(J(q)) < \min(m, n) \quad \Leftrightarrow \quad \sigma_{\min}(J) = 0$$

El robot **pierde grados de libertad** en el espacio de tarea.

Tipos en robots seriales:

1. **Singularidad de frontera**
   Brazo completamente extendido ($d = L_1 + L_2$) o plegado
2. **Singularidad interior**
   Configuración en la que columnas de J son linealmente dependientes
3. **Singularidad de muñeca**

Detección con SVD

$$J = U\Sigma V^\top, \quad \Sigma = \text{diag}(\sigma_1, \ldots, \sigma_r)$$

- $\sigma_{\min} \approx 0 \Rightarrow$ cerca de singularidad
- Índice de condición: $\kappa = \sigma_{\max}/\sigma_{\min}$

Robot 2R – caso explícito

$$\det(J) = -L_1 L_2 \sin q_2$$

Singular cuando $q_2 = 0°$ (ext.) o $180°$ (pleg.)
$\Rightarrow$ la velocidad en y no se puede controlar

DH $\rightarrow$ Jacobiano Geométrico

La tabla DH permite calcular el **Jacobiano geométrico** de posición directamente:

$$J_{\text{pos}} = \left[\, \mathrm{z}_0 \times (\mathrm{p}_n - \mathrm{p}_0) \;\middle|\; \mathrm{z}_1 \times (\mathrm{p}_n - \mathrm{p}_1) \;\middle|\; \cdots \;\middle|\; \mathrm{z}_{n-1} \times (\mathrm{p}_n - \mathrm{p}_{n-1}) \,\right]$$

donde, para cada marco i extraído de ${}^0T_i$:

$$\mathrm{z}_i = {}^0T_i[0{:}3,\, 2], \qquad \mathrm{p}_i = {}^0T_i[0{:}3,\, 3]$$

Conexión con la cinemática diferencial

$$\dot{\mathrm{x}} = J(q)\,\dot{q}$$

$$\boldsymbol{\tau} = J^\top(q)\,\mathrm{F}$$

El Jacobiano *traduce* velocidades **y** fuerzas entre el espacio articular y el espacio de tarea.

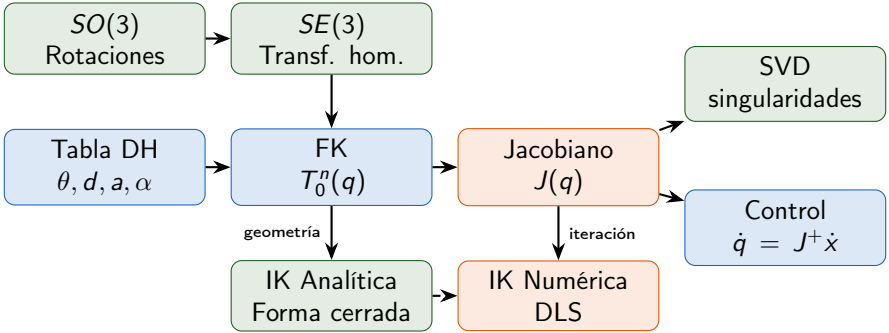
Control en espacio de tarea

$$\dot{q} = J^+\dot{\mathrm{x}}_d$$

$$\dot{q} = J^+\dot{\mathrm{x}}_d + (I - J^+J)\dot{q}_0$$

La segunda fórmula: control redundante.
$(I - J^+J)$: proyector sobre el espacio nulo de J.

Mapa conceptual: Rotaciones → DH → IK



Mensaje central

Las **rotaciones** son el fundamento; la **tabla DH** las organiza para la cadena cinemática completa: FK → Jacobiano → IK → Control.

Comparativa FK vs. IK

Propiedad	Cinemática Directa (FK)	Cinemática Inversa (IK)
Dirección	$q \to \mathrm{x}$	$\mathrm{x} \to q$
Unicidad	Siempre única	Múltiple o ninguna
Complejidad	$O(n)$ – eficiente	NP en general
Herramienta	Multiplicación DH	Analítica, DLS, NN
Singularidades	No aplica	Problemáticas
Redundancia	No aplica	Espacio nulo de J
Uso típico	Simulación, visualización	Planeación, control

Regla práctica

- Usa **analítica** cuando el robot tiene $\leq$ 3-DOF o estructura especial (SCARA, muñeca esférica)
- Usa **DLS** para robots arbitrarios, redundantes o cerca de singularidades

Puntos clave de la sesión

Rotaciones y $SE(3)$

- $R \in SO(3)$: $R^\top R = I$, $\det R = 1$
- RPY, eje-ángulo, cuaterniones: equivalentes
- $T \in SE(3)$: une R y traslación p
- Composición no conmutativa

Formalismo DH

- 4 parámetros (θ, d, a, α) por eslabón
- Orden estricto: $R_z \to T_z \to T_x \to R_x$
- $T_0^n = T_0^1 \cdots T_{n-1}^n$

IK analítica

- Ley de cosenos + atan2
- Dos soluciones en 2R: codo $\uparrow\downarrow$
- Desacoplamiento en SCARA

IK numérica (DLS)

$$\Delta q = J^\top (JJ^\top + \lambda^2 I)^{-1} \mathrm{e}$$

- Jacobiano derivado de la tabla DH
- Robusto frente a singularidades
- Convergencia cuadrática

Referencias recomendadas

Libros de texto:

- Siciliano, Sciavicco, Villani, Oriolo — *Robotics: Modelling, Planning and Control*, Springer 2009
- Spong, Hutchinson, Vidyasagar — *Robot Modeling and Control*, Wiley 2020
- Craig, J.J. — *Introduction to Robotics: Mechanics and Control*, Pearson 2005
- Murray, Li, Sastry — *A Mathematical Introduction to Robotic Manipulation*, CRC 1994

Recursos en línea:

- Peter Corke — *Robotics, Vision and Control* + Robotics Toolbox (Python)
- `pytransform3d`: cuaterniones, $SO(3)$, $SE(3)$ en Python
- `roboticstoolbox-python`: DH, FK, IK, Jacobiano
- `scipy.spatial.transform`: RPY, cuaterniones, Rodrigues

¿Preguntas?

Prof. Alberto Muñoz

amunoz@tec.mx

TE3001B — Tecnológico de Monterrey

Oficina: http://tinyurl.com/MunozTec